

# FolderGuardian

A real-time file monitoring and AI document automation system built with Python, LangChain, LangGraph, and RAG.

## 1. What the Project Is

FolderGuardian is a desktop application that watches up to four folders for incoming documents and turns them into finished, structured reports with no manual writing involved. It combines three ideas in one tool:

- **Real-time monitoring** — a Watchdog observer detects new files the moment they appear, resolves copy-vs-move races, and batches files that arrive together.
- **Human gatekeeping** — a desktop popup shows each incoming file's metadata (type, size, word count) and lets the user **Allow** or **Deny** it before any processing happens. Denied files are returned to their origin or deleted.
- **Agentic AI processing** — allowed files enter a LangGraph state-machine pipeline that ingests the document, builds a searchable vector knowledge base (RAG), and generates a professional DOCX report via a Groq-hosted LLM (llama-3.1-8b-instant by default).

The user chooses a **2-step** (Ingest → Generate) or **4-step** (Ingest → Analyze → Generate → Review) workflow per file. All pipeline work runs on a background thread while a live Tkinter popup streams progress, and every artifact — the original file, JSON summaries, the vector index, and the final DOCX — is filed into organized per-document folders.

## 2. Architecture

FolderGuardian follows a three-stage design: **monitoring & gatekeeping** (main.py), an **agentic LangGraph pipeline** (v3/), and a local **artifact store** (knowledge/, per-document step folders).

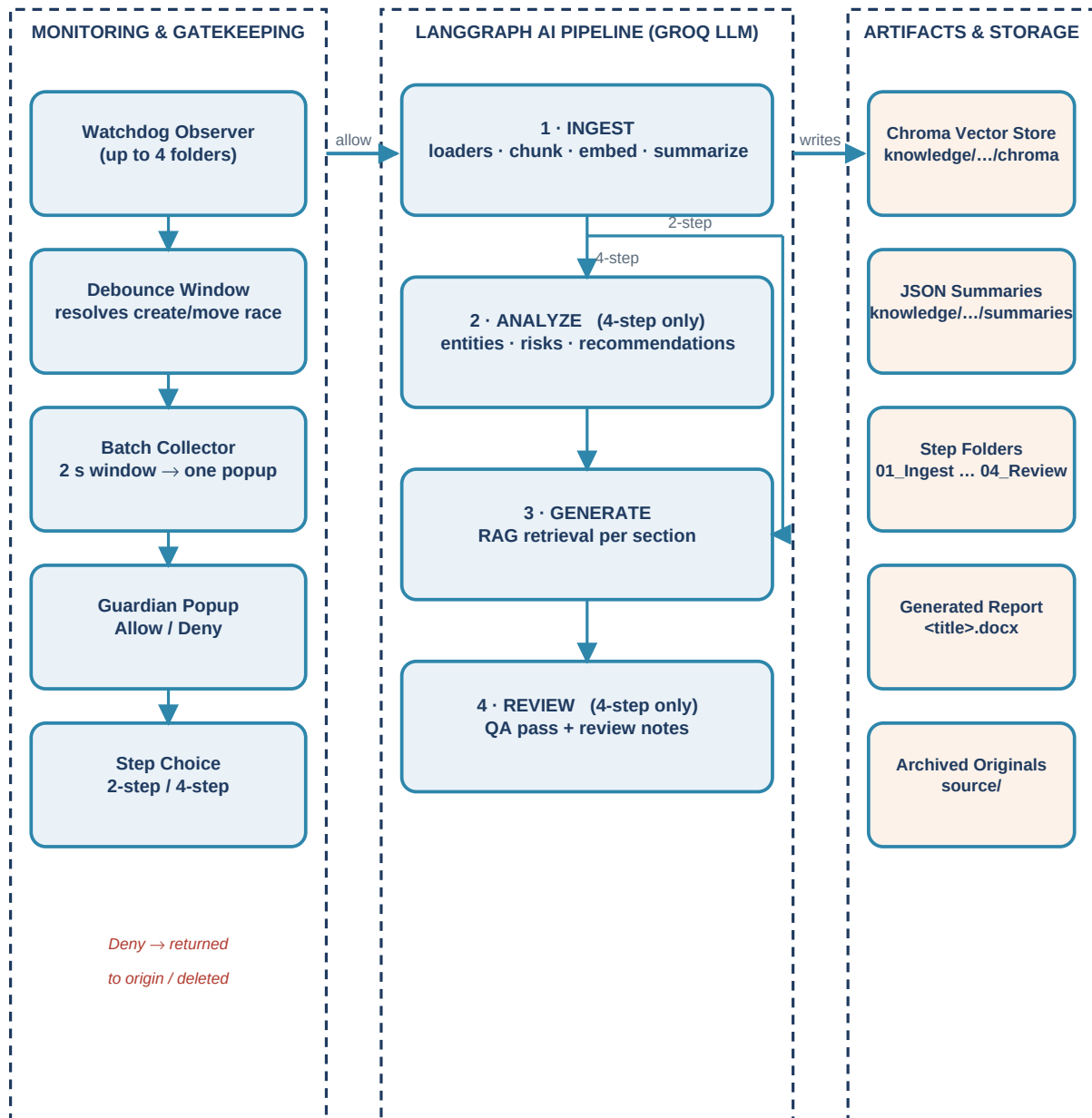


Figure 1 — FolderGuardian end-to-end architecture

## 2.1 Key Components

| Component                     | File(s)                                             | Responsibility                                                              |
|-------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------|
| Watchdog observer & debouncer | main.py                                             | Watches folders, resolves macOS create→move races with a 1 s pending        |
| Batch collector & dispatcher  | main.py                                             | Groups files arriving within 2 s into one popup; sequences all Tkinter popu |
| Guardian UI                   | popup.py, step_choice_popup.py, allow_deny_popup.py | Allow/Deny popup, 2-step/4-step chooser, live processing progress popup.    |
| LangGraph pipeline            | v3/graph.py, v3/pipeline.py                         | Compiles a 2- or 4-node state graph; runs it on a background thread with    |
| Ingest node                   | v3/nodes/ingest.py                                  | Parses 15+ formats, chunks + embeds into Chroma (docs > 12 000 chars        |
| Analyze node (4-step)         | v3/nodes/analyze.py                                 | Extracts key entities, risks, and recommendations as JSON.                  |
| Generate node                 | v3/nodes/generate.py                                | Retrieves per-section context from RAG, writes each section, builds the D   |
| Review node (4-step)          | v3/nodes/review.py                                  | LLM QA pass; saves review notes and the final reviewed DOCX.                |

|            |                                      |                                                                       |
|------------|--------------------------------------|-----------------------------------------------------------------------|
| RAG store  | v3/rag/store.py, v3/rag/summaries.py | Chroma + HuggingFace all-MiniLM-L6-v2 embeddings, persisted per wat   |
| File tools | file_info.py, v3/tools/              | Metadata extraction (rows, words, pages), document readers, DOCX writ |

## 2.2 Data Flow of One Document

- **Step 1.** A file lands in a watched folder; Watchdog fires and the debouncer commits it to the queue once the create/move race is resolved.
- **Step 2.** Files arriving within 2 s are batched; one Guardian popup lists them with metadata. The user allows or denies each.
- **Step 3.** For an allowed file, the user picks a 2-step or 4-step workflow, and the processing popup starts streaming live progress.
- **Step 4.** Ingest: the document is parsed (TXT, MD, CSV, JSON, XML, HTML, PDF, DOCX, XLSX), chunked (1 000 chars, 150 overlap), embedded, indexed into a per-folder Chroma store, and summarized into structured JSON by the LLM.
- **Step 5.** Analyze (optional): the LLM extracts entities, risks, and recommendations.
- **Step 6.** Generate: for every configured section (Overview, Key Procedures, Important Details, Action Items by default), the pipeline retrieves the top-4 relevant chunks and the LLM writes the section grounded in that context.
- **Step 7.** Review (optional): an LLM QA pass writes review notes and copies the finished DOCX into the Review folder.
- **Step 8.** The original file is archived under source/; the finished report and all intermediates are filed in per-document workflow folders.

Small documents ( $\leq 12\,000$  characters) skip the vector store and are passed directly to the LLM context window — an optimization that avoids unnecessary embedding work. The pipeline thread has a 180-second timeout and reports every error back through the UI popup.

### 3. Real-World Use Cases

Because FolderGuardian is format-agnostic and folder-driven, it slots into any workflow where documents arrive continuously and someone has to read, summarize, and report on them.

#### Accounts Payable — Invoice Processing

Invoices dropped by email or scanner into a shared Accounting/Inbox folder are summarized (vendor, amounts, due dates), analyzed for risks such as duplicate charges, and turned into a standardized approval report. The Allow/Deny gate keeps personal files that were copied to the wrong place out of the financial pipeline.

#### Legal & Compliance — Contract Intake

Incoming NDAs and vendor contracts are ingested into a searchable knowledge base so the team can ask 'which contracts mention indemnification?' The 4-step workflow adds a risk analysis (unusual liability clauses) and a QA review note for the paralegal before the summary report is circulated.

#### IT Operations — Report Triage

Nightly exports (CSV logs, JSON metrics, XLSX reports) from monitoring tools are dropped into a watched folder. FolderGuardian summarizes each one, flags anomalies in the Analyze step, and produces a morning briefing DOCX for the ops stand-up — no human has to open 20 spreadsheets.

#### Research & Academia — Literature Intake

PDFs of papers collected by a research group are automatically summarized into overview/key-topics JSON artifacts and indexed for RAG, letting students generate literature-review drafts grounded in the actual papers rather than guesswork.

#### HR — Candidate Document Handling

Resumes and portfolio DOCX/PDF files arriving from a job-board drop folder are reviewed (denied files never touch the pipeline), summarized into a consistent candidate profile, and generated into a shortlist report for the hiring manager.

#### Field Services — Equipment Manual Digitization

Scanned or downloaded equipment manuals (like the bundled sample\_pump\_manual.txt) are ingested once into the knowledge base; the Generate step then produces maintenance quick-reference cards and action-item lists that technicians can keep on a tablet.

#### Small Business — Knowledge Base Builder

Every SOP, policy, or how-to document a team saves into a shared folder quietly becomes part of a growing, searchable knowledge base with a generated summary — turning a messy shared drive into an onboarding asset.

### 4. Technology Stack

| Layer | Technology |
|-------|------------|
|-------|------------|

|                     |                                                                      |
|---------------------|----------------------------------------------------------------------|
| Monitoring          | Watchdog (cross-platform filesystem observer)                        |
| UI                  | Tkinter desktop popups (allow/deny, workflow choice, live progress)  |
| Agent orchestration | LangGraph StateGraph with conditional edges                          |
| LLM                 | Groq API — llama-3.1-8b-instant (configurable via GROQ_MODEL)        |
| Framework           | LangChain + langchain-groq + community loaders                       |
| RAG                 | Chroma vector store, HuggingFace all-MiniLM-L6-v2 embeddings         |
| Documents           | pypdf, python-docx, openpyxl (PDF / DOCX / XLSX)                     |
| Runtime             | Python 3.13+; API key supplied via GROQ_API_KEY environment variable |

*Repository: [github.com/Karthi-64/Capstone\\_Project](https://github.com/Karthi-64/Capstone_Project) — report generated September 2026 from the V3 codebase.*